



合肥工业大学

计算机与信息学院

计算机科学与技术专业新生研讨课

课程报告

姓 名： 李畅

专业班级： 计算机科学与技术 23-1 班

学 号： 2023212157

2024 年 01 月 03 日

目录

1	R 语言简介及安装	- 3 -
1.1	R 语言的历史	- 3 -
1.2	R 语言的特点	- 3 -
1.3	R 语言的下载及安装	- 3 -
1.4	RStudio 介绍及安装	- 5 -
1.5	R 扩展软件包	- 5 -
1.6	帮助	- 6 -
2	R 的基本操作	- 7 -
2.1	赋值与注释	- 7 -
2.2	输出	- 7 -
2.3	运算符	- 7 -
2.4	数学函数	- 8 -
2.5	复合函数	- 9 -
2.6	基础绘图	- 9 -
3	R 中的数据结构	- 11 -
3.1	向量	- 11 -
3.2	标量	- 11 -
3.3	矩阵	- 12 -
3.4	数组	- 13 -
3.5	数据框	- 13 -
3.6	列表	- 14 -
3.7	因子	- 15 -
3.8	本章总结	- 16 -
4	利用 R 进行基础回归分析	- 17 -
4.1	最小二乘回归法 (OLS)	- 17 -
4.2	利用 OLS 进行简单线性回归	- 18 -
4.3	利用 OLS 进行多项式回归	- 20 -
4.4	利用 OLS 进行带交互项的多元线性回归	- 21 -
5	在 R 中通过机器学习技术生成预测模型	- 23 -
5.1	数据准备	- 23 -
5.2	Logistic 回归	- 24 -
5.3	决策树	- 24 -
5.4	支持向量机	- 25 -
6	写在最后	- 26 -
	参考资料	- 27 -

1 R 语言简介及安装

1.1 R 语言的历史

R 语言以 S 语言为基础。S 语言由 Rick Becker、John Chambers、Alan Wilks 等人在贝尔实验室共同研发。值得一提的是，贝尔实验室还开发了著名的 C 语言和 Unix 系统。R 语言最初由新西兰 Auckland 大学的 Ross Ihaka 和 Robert Gentleman 于 1997 年发布，它实现了与 S 语言基本相同的功能和统计功能。R 语言由 R 核心团队负责开发，但全球的用户都可以为其贡献软件包，使其成为一个真正开放和多元化的项目。

1.2 R 语言的特点

- 与大多数统计软件如 Matlab、SPSS 相比，R 语言是一款免费、开源的软件，无需支付昂贵的费用来使用。
- R 语言可以在多种平台上运行，如 Windows、macOS、UNIX 上均可运行。
- R 语言具有完善的数据类型，如向量、矩阵、因子、数据集等，并且支持缺失值。
- R 语言的功能可以被集成到其他语言编写的应用程序中，如 C++、Java 等。
- R 语言提供了多种多样的数据分析技术，如线性回归、时间序列分析等。
- R 语言拥有顶尖的制图功能，可以将复杂的数据进行可视化。
- R 语言有上万扩展包，可用性强。

1.3 R 语言的下载及安装

R 的官方网站在 <https://www.r-project.org/>。

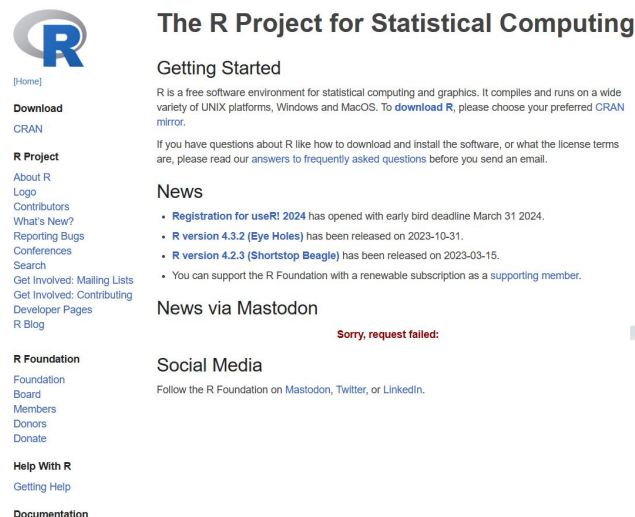


图 1 R 的官方网站

点击上方 R 的官方网站的 **download R**，我们可以看到 R 的许多镜像网站 (CRAN mirrors)，我们在国内，选用位于中国的镜像网站下载更快。

China	https://mirrors.tuna.tsinghua.edu.cn/CRAN/ https://mirrors.bfsu.edu.cn/CRAN/ https://mirrors.pku.edu.cn/CRAN/ https://mirrors.ustc.edu.cn/CRAN/ https://mirrors.zju.edu.cn/CRAN/ https://mirror-hk.koddos.net/CRAN/ https://mirrors.e-ducation.cn/CRAN/ https://mirrors.glu.edu.cn/CRAN/ https://mirror.lzu.edu.cn/CRAN/ https://mirrors.nju.edu.cn/CRAN/ https://mirrors.sjtu.edu.cn/cran/ https://mirrors.sustech.edu.cn/CRAN/ https://mirrors.nwafu.edu.cn/cran/	TUNA Team, Tsinghua University Beijing Foreign Studies University Peking University University of Science and Technology of China Zhejiang University KoDDoS in Hong Kong Elite Education Qilu University of Technology Lanzhou University Open Source Society eScience Center, Nanjing University Shanghai Jiao Tong University Southern University of Science and Technology (SUSTech) Northwest A&F University (NWAFU)
-------	---	---

图 2 位于中国的镜像网站

我们选择一个位于中国的镜像网站，点击进入后页面如下。

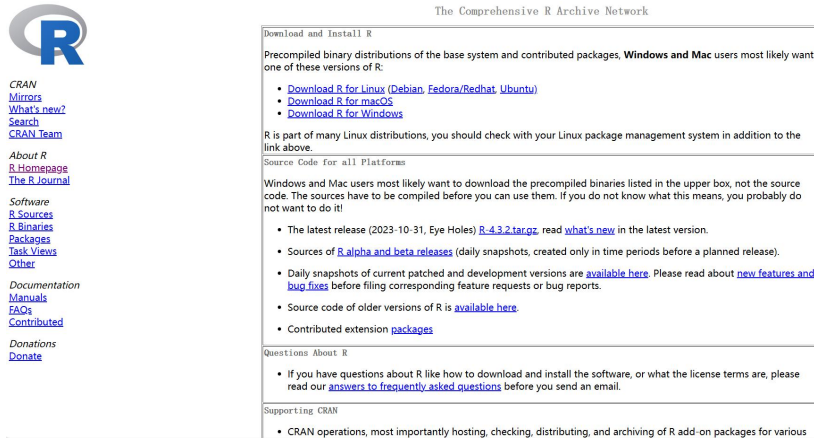


图 3 R 的镜像网站

我们点击 **Download R for Windows**，进入 **Windows** 系统的 R 的下载页面。除了 **base** 为 R 的安装程序，还有 **contrib** 为 R 附加的扩展软件包下载链接，以及 **Rtools** 链接，是在 R 中调用 C、C++ 和 Fortran 等程序代码时需要用的编译工具。这里我们选择最新的 4.3.2 版本进行安装。注意 **Rtools** 的版本下载时要选择与 R 的版本一致。

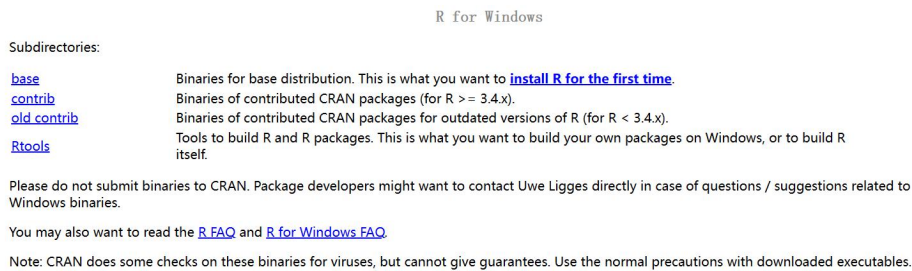


图 4 R for Windows

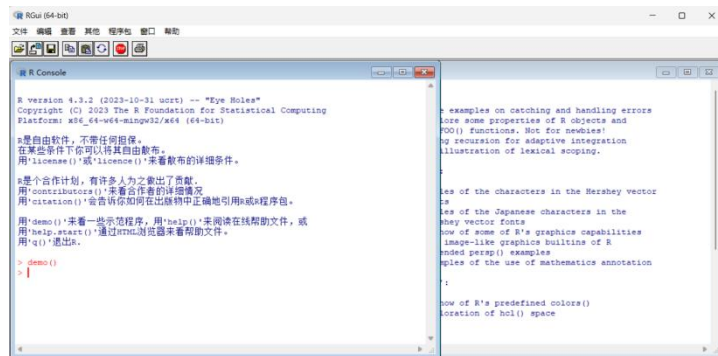


图 5 RGUI 界面

在 Windows 操作系统中的 R 软件有一个 R GUI 软件，即图形窗口模式的 R 软件，如图 5 所示。

1.4 RStudio 介绍及安装

R 语言的标准界面十分简单，仅仅针对输入代码行使用，在真实的项目中，往往需要一个更加全面的集成开发环境来编辑代码和查看输出结果。对于 R 语言来说，这样的集成开发环境有 Visual Studio for R、RStudio Desktop 等，RStudio Desktop 便是迄今最受欢迎的一款工具。

RStudio 官网为 <https://www.rstudio.com>，在官网上下载安装后启动呈现如图 6 所示的界面。左上方的部分为脚本窗口，程序源代码在这里显示；左下角的部分为控制台窗口，程序在这里运行；右上角为环境窗口，这里会列出已经创建了的对象；右下方为绘图窗口，绘制的图像会在这里显示。

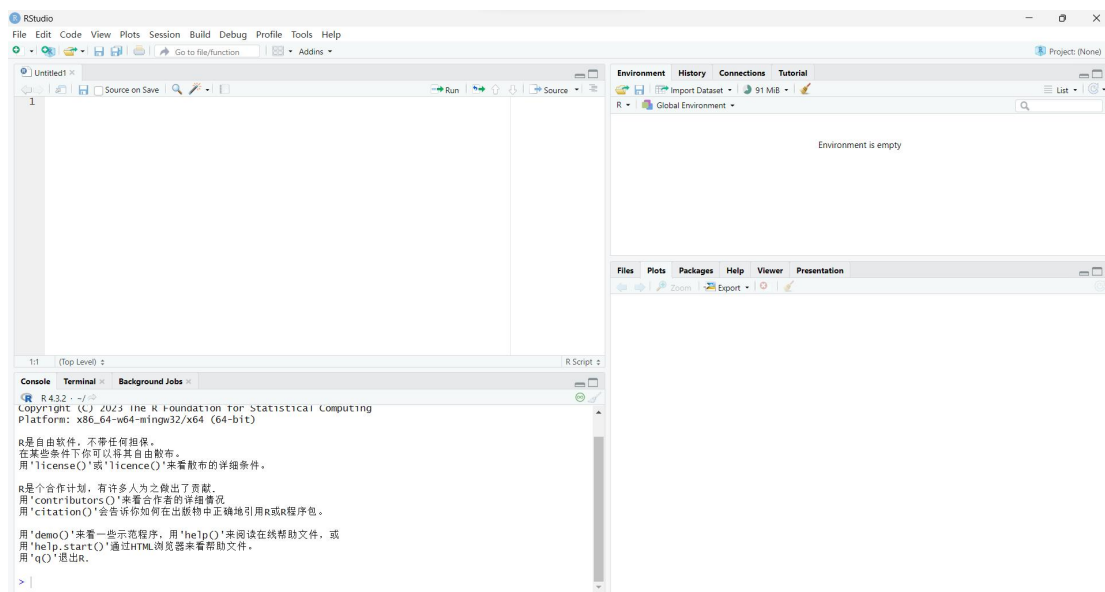


图 6 RStudio 界面

1.5 R 扩展软件包

R 有一万多个扩展软件包(package)，提供了各种各样的功能扩展。包是 R 函数、数据、预编译代码以一种定义完善的格式组成的集合，计算机上存储包的目录称为库(library)。函数 `libPaths()` 能够显示库所在的位置，函数 `library()` 则可以显示库中有哪些包。

在安装基本 R 软件时，已经伴随安装了一些必要的扩展包，如 `base`, `datasets`, `utils`, `stats`, `graphics` 等，这些包在启动 R 时会默认载入，使用命令 `search()` 可以告诉我们哪些包已经被加载并且已经可以被载入其他包我们需要使用函数 `library()` 来载入这个包，例如载入 `tidyverse` 包我们需要下面的命令：
`library(tidyverse)`

在 R 语言中第一次安装一个包需要使用命令 `install.packages()`，例如安装包 `gclus` 时需要使用以下命令：
`Install.packages(gclus)`

一个包仅需要安装一次，但这些包经常会被他们的作者更新，这时我们使用命令 `installed.packages()` 来更新，例如：

Installed.packages(gclus)

在 RStudio 中我们也有另一种更为简便的方式来安装扩展包，在 RStudio 中调用用“Tools”菜单的“Install Packages”，输入或者在列表中所需要安装的扩展包就可以安装该扩展包。

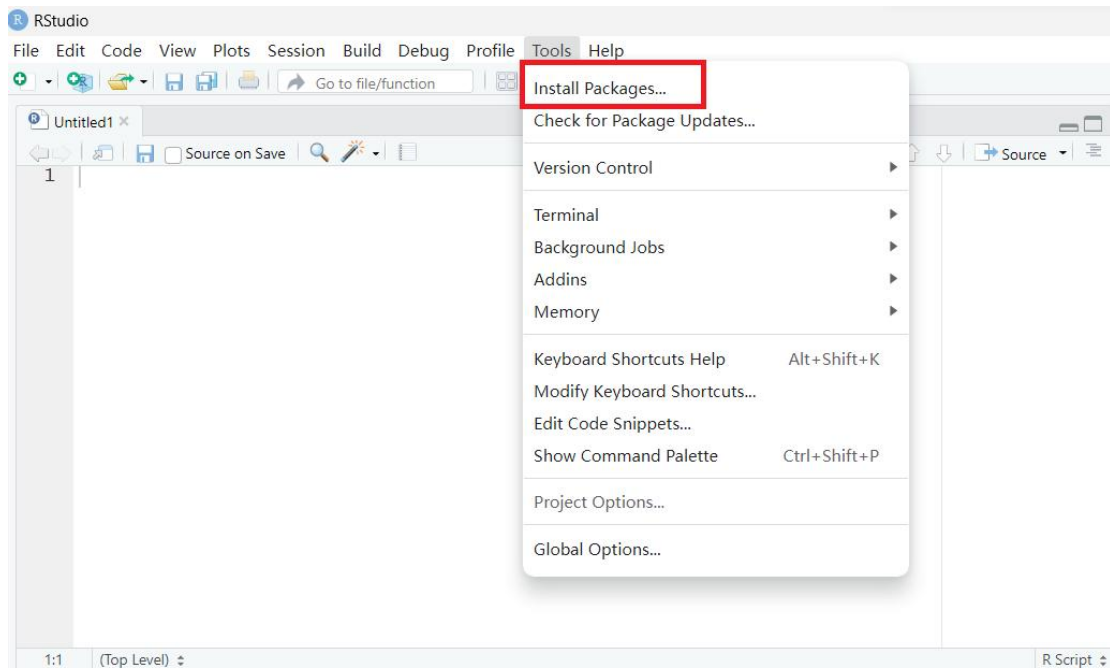


图 7 在 RStudio 中安装扩展包

1.6 帮助

我们可以使用以下命令来从 R 语言中获取帮助：

```
#输出通用的帮助信息
help.start()
#输出函数 foo()的帮助信息
help("foo")或?foo
#在帮助系统中查找名称中带有字段 foo 的实例(包、类、函数等)的帮助信息
Help.search("foo")或??foo
#输出 foo 包的帮助信息
help(package="foo")
#输出函数 foo()的示例信息，可省略引号
example("foo")
#列出当前已经加载的包中所有可用的示例信息集
data()
```

2 R 的基本操作

2.1 赋值与注释

R 语言以大于号(">")为提示符(如图 5 中所示),在提示符后面键入命令,命令的文字型结果马上显示在命令下方,命令的图形结果单独显示在一个图形窗口中。

R 语言的语句由函数和赋值构成。在 R 语言中,使用<-为对象赋值(R 语言也允许使用=为对象赋值,但它不是 R 语言的标准语法,一些情况下使用=赋值会出现问题)。R 语言也可以反转赋值方向,->则表示把符号左边的值赋给符号右边。例如,语句

```
x <- 5
```

表示把 5 的值赋给对象 x。而语句

```
5 -> x
```

也表示同样的效果。

R 语言的注释由#开头,在#之后出现的所有的文本都会被 R 的解释器忽略。

#以下语句会被 R 语言忽略

注意在输入命令时关闭中文输入法,否则输入一些中文标点将导致程序运行错误。

2.2 输出

print()函数可以显示一个表达式的结果,如

```
print(sin(pi/6))
```

输出结果如下:

```
> print(sin(pi/6))  
[1] 0.5
```

cat()函数显示多项内容,包括数值和文本,文本包含在引号中,如

```
cat("sin(pi/6)=",sin(pi/6),"\\n")
```

输出结果如下:

```
> cat("sin(pi/6)=",sin(pi/6),"\\n")  
sin(pi/6)= 0.5
```

其中\\n 为转义字符,表示换行。

除此之外,通过文件输出时,我们还可以使用 write.table(),write.csv()等函数。

2.3 运算符

R 语言中常见的一些算数运算符和逻辑运算符如下表所示:

运算符	描述	运算符	描述
+	加	^ 或 **	求幂
-	减	x %% y	求余
*	乘	x %/% y	整数除法
/	除		如 5%/%2=2

表 1 R 语言中常见的算术运算符

运算符	描述	运算符	描述
<	加	!=	不等于
<=	减	!x	非 x
>	乘	x y	x 或 y
>=	除	x&y	x 和 y
==	严格相等	isTrue(x)	x 是否为真

表 2 R 语言中常见的逻辑运算符

2.4 数学函数

以下为 R 语言中常用的基本数学函数：

函数	描述
sqrt()	求平方根
exp()	求 e 的幂
log()	求自然对数
log10()	求以 10 为底数的对数
round(x,y)	取整函数，把 x 四舍五入到 y 位小数（y 可省略）
floor(x,y)	取整函数，把 x 向下取整（y 可省略）
ceiling(x,y)	取整函数，把 x 向上取整（y 可省略）
pi	圆周率
sin()	正弦函数，自变量为弧度制
cos()	余弦函数，自变量为弧度制
tan()	正切函数，自变量为弧度制
asin()	反正弦函数，结果为弧度制
acos()	反余弦函数，结果为弧度制
atan()	反正切函数，结果为弧度制

表 3 R 语言中常用基本数学函数

例如以下运行结果：

```
> sqrt(2)
[1] 1.414214
> exp(2)
[1] 7.389056
> log(2)
[1] 0.6931472
> log10(1000)
[1] 3
> round(1.23456,3)
[1] 1.235
> floor(1.5)
[1] 1
> ceiling(1.5)
[1] 2
> pi
[1] 3.141593
> sin(pi/6)
[1] 0.5
> cos(pi/6)
[1] 0.8660254
> tan(pi/4)
[1] 1
> asin(1)
[1] 1.570796
> acos(0.5)
[1] 1.047198
> atan(1)
[1] 0.7853982
```

图 8 基本数学函数运行结果

2.5 复合函数

R 语言中可以直接通过与数学运算类似的方式来进行复合函数运算，如：

```
log(exp(1))
```

表示先求自然常数的一次幂，然后再对该数据求自然对数，运行结果如下：

```
> log(exp(1))  
[1] 1
```

在 R 语言中也可以通过管道运算符(`|>`)来实现对于复合函数的多层调用，如 `f3(f2(f1(x)))` 写成 `x|>f1()|>f2()|>f3()` 就更容易看清楚数据的流向，上面的函数我们可以改写成下面的形式：

```
exp(1) |> log()
```

运行结果如下：

```
> exp(1)|>log()  
[1] 1
```

2.6 基础绘图

使用函数 `curve()` 可以进行曲线的绘制，`curve()` 函数第一自变量为函数解析式，第二、第三自变量为函数绘图区间，同时可以使用 `abline()` 函数添加参考线，例如函数

```
curve(sin(x), 0, 2*pi)  
abline(h=0)
```

可以来绘制函数 $y=\sin(x)$ 在区间 $[0, 2\pi]$ 的图像，并添加了 $y=0$ 处的参考线，其绘制结果如下：

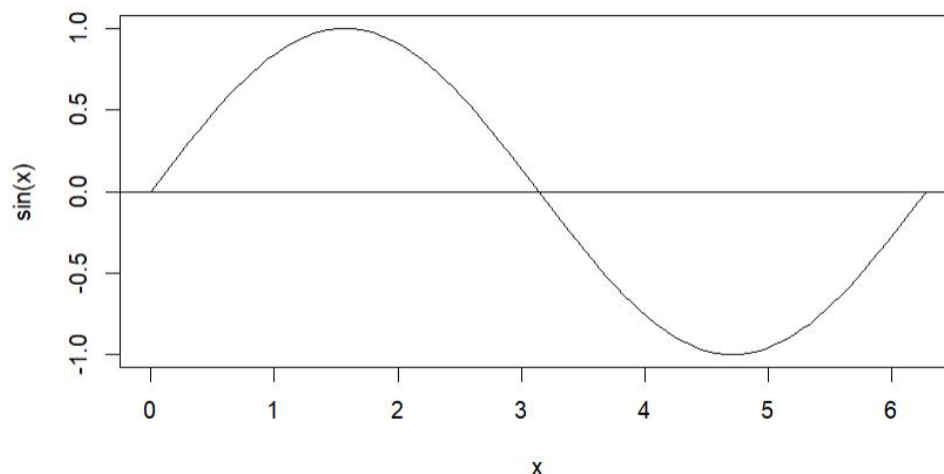


图9 函数 $y=\sin(x)$ 的图像

利用函数 `plot()` 可以进行散点图的绘制，`plot()` 函数的第一个自变量是各点的横坐标，第二个自变量是各点的纵坐标，例如函数：

```
plot(1:10, sqrt(1:10))
```

绘制的散点图如下：

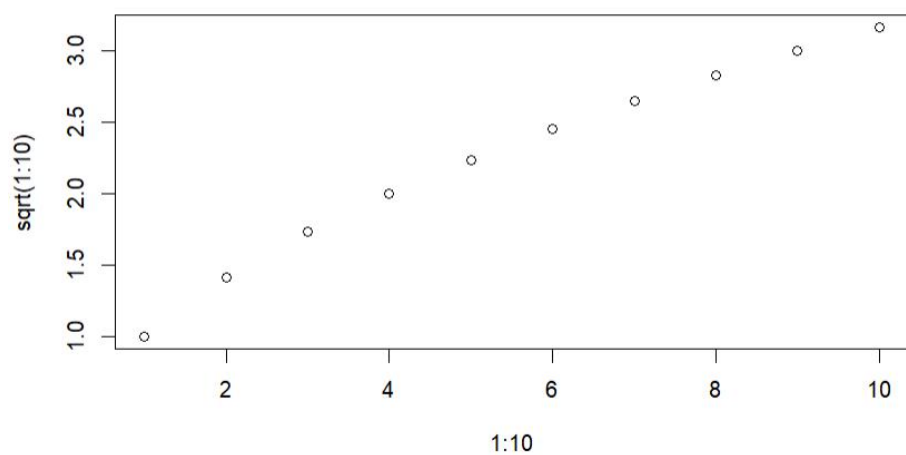


图 10 绘制的散点图的图像

函数 `barplot()` 可以进行条形统计图的绘制，例如下面的函数：

```
barplot(c("男生"=39, "女生"=7), main="计科一班班级人数统计")
```

绘制的条形图如下：

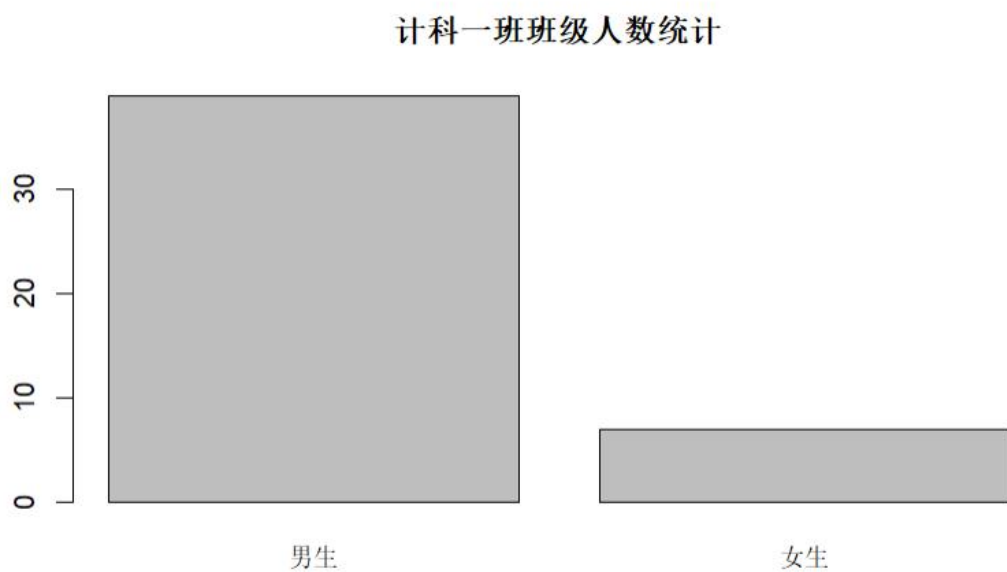


图 11 绘制的条形图的图像

除此之外，R 语言中还提供了多种多样的绘图函数以及图像的控制方式，后面会使用到。

3 R 中的数据结构

3.1 向量

向量(vector), 是用于存储数值型、字符型或者逻辑型的一维数组。函数 `c()` 可以执行组合的功能, 可以使用该函数来创建向量。R 语言中向量可以有数值型、字符型或者逻辑型, 单个向量中的数据必须是相同的类型。例如:

```
a <- c(1, 2, 3, 4, 5)
b <- c("one", "two", "three")
c <- c(TRUE, FALSE, TRUE)
```

a 是数值型变量, b 是字符型变量, c 是逻辑型变量。

我们也可以使用冒号的形式填充一个向量, 例如:

```
a <- c(1:5)
a <- c(1, 2, 3, 4, 5)
```

上面两种形式是等价的。

通过在方括号中给定元素所处位置的数值, 我们可以访问向量中的元素。与 `c++` 不同, R 语言的下标从 1 开始, 而非从 0 开始。例如下列语句:

```
a <- c("q", "w", "e", "r")
a[3]
```

输出结果为下标 3 对应的元素 e, 如下图所示:

```
> a <- c("q", "w", "e", "r")
> a[3]
[1] "e"
```

`length(x)` 可以求向量 x 的长度。长度为零的向量表示为 `numeric(0)`。`numeric()` 函数可以用来初始化一个指定元素个数而元素都等于零的数值型向量, 如 `numeric(10)` 会生成元素为 10 个零的向量。

R 中的函数一般都是向量化的: 在 R 中, 如果普通的一元函数以向量为自变量, 一般会对每个元素计算。这样的函数包括 `sqrt()`, `log10()`, `log()`, `exp()`, `sin()`, `cos()`, `tan()` 等许多。例如:

```
a <- c(1, 4, 9, 16, 25)
sqrt(a)
```

输出结果为:

```
> a <- c(1, 4, 9, 16, 25)
> sqrt(a)
[1] 1 2 3 4 5
```

如果自己编写的函数没有考虑向量化问题, 可以用 `Vectorize()` 函数将其转换成向量化版本。

3.2 标量

标量(scalar), 是只含有一个元素的向量, R 没有单独的标量类型, 标量实际是长度为 1 的向量。向量与标量的运算为每个元素与标量的运算, 例如:

```
x <- c(3, 5)
x+2
```

输出结果为:

```
[1] 5 7
```

3.3 矩阵

矩阵(matrix), 是一个二维数组, 并且每个元素全部都具有相同的数据类型(数值型、字符型或者逻辑型), 一般采取以下形式创建矩阵:

```
Mymatrix <- matrix(vector, nrow=number_of_rows,
                    ncol=number_of_columns, byrow=logical_value,
                    dimnames=list(rowname, colname))
```

其中, `vector` 包含了矩阵的元素, `nrow` 和 `ncol` 用来指定行和列的维数默认情况下矩阵按列填充, `dimnames` 包含了可选的、以字符型向量表示的行名和列名, 选项 `byrow` 表示矩阵按行填充(`byrow=TRUE`)或按列填充(`byrow=FALSE`)。例如:

```
x <- matrix (1:15, nrow=3, ncol=5, byrow=TRUE)
```

表示的矩阵如下图所示:

```
> x <- matrix (1:15, nrow=3, ncol=5, byrow=TRUE)
> x
      [,1] [,2] [,3] [,4] [,5]
[1,]    1    2    3    4    5
[2,]    6    7    8    9   10
[3,]   11   12   13   14   15
```

通过对应的下标可以访问对应的行或列。

通过函数 `solve(A)` 可以求 A 的逆矩阵, 例如:

```
y <- matrix (1:4, nrow=2, ncol=2, byrow=TRUE)
y
solve(y)
```

输出的逆矩阵如下图所示:

```
> y <- matrix (1:4, nrow=2, ncol=2, byrow=TRUE)
> y
      [,1] [,2]
[1,]    1    2
[2,]    3    4
> solve(y)
      [,1] [,2]
[1,] -2.0  1.0
[2,]  1.5 -0.5
```

通过函数 `solve(A,b)` 可以求线性方程组 $Ax = b$ 中的 x 的值, 例如:

```
y <- matrix (1:4, nrow=2, ncol=2, byrow=TRUE)
solve(y,z(1,2))
```

求解结果为:

```
> y <- matrix (1:4, nrow=2, ncol=2, byrow=TRUE)
> solve(y,c(1,2))
[1] 0.0 0.5
```

通过函数 `apply(A, 2, FUN)` 把矩阵 A 的每一列分别输入到函数 FUN 中, 得到对应于每一列的结果, 例如:

```
y <- matrix (1:4, nrow=2, ncol=2, byrow=TRUE)
apply(y, 2, sqrt)
```

结果为:

```
> y <- matrix(1:4, nrow=2, ncol=2, byrow=TRUE)
> apply(y, 2, sqrt)
      [,1]      [,2]
[1,] 1.000000 1.414214
[2,] 1.732051 2.000000
```

3.4 数组

数组(array), 是维度大于 2 的矩阵。与矩阵类似, 数组可以通过函数 `array()` 创建, 形式如下:

```
myarray <- array(vector, dimensions, dimnames)
```

其中: `vector` 包含了数组中的数据, `dimensions` 是一个数值型的向量, 表示了各维度下标的最大值, `dimnames` 是可选的, 用来表示各维度的名称。例如:

```
dim1 <- c("A1")
dim2 <- c("B1", "B2")
dim3 <- c("C1", "C2", "C3")
myarray <- array(1:6, c(1, 2, 3), dimnames=list(dim1, dim2, dim3))
myarray
```

结果为:

```
> dim1 <- c("A1")
> dim2 <- c("B1", "B2")
> dim3 <- c("C1", "C2", "C3")
> myarray <- array(1:6, c(1, 2, 3), dimnames=list(dim1, dim2, dim3))
> myarray
, , C1
    B1 B2
A1  1  2

, , C2
    B1 B2
A1  3  4

, , C3
    B1 B2
A1  5  6
```

数组的其他操作与矩阵类似。

3.5 数据框

数据框类似于一个矩阵, 有个横行、个纵列, 但各列允许有不同类型: 数值型向量、因子、字符型向量、日期时间向量。同一列的数据类型相同。在 R 中数据框是一个特殊的列表, 其每个列表元素都是一个长度相同的向量。数据框可以通过以下方式创建:

```
mydata <- data.frame(col1, col2, col3, ...)
```

其中, 列向量 `col1, col2, col3` 等可以为字符型、逻辑型、字符型等的的数据。

下面的语句创建了一个数据框:

```
name <- c("Alan", "Jack", "Lucy")
age <- c(18, 19, 18)
studentID <- c(20232121, 20232122, 20232123)
student_data <- data.frame(name, age, studentID)
student_ID
```

其运行结果如下：

```
> name <- c("Alan", "Jack", "Lucy")
> age <- c(18, 19, 18)
> studentID <- c(20232121, 20232122, 20232123)
> student_data <- data.frame(name, age, studentID)
> student_data
  name age studentID
1 Alan  18  20232121
2 Jack  19  20232122
3 Lucy  18  20232123
```

数据框不能作为矩阵参加矩阵运算。需要时可以用 `as.matrix()` 函数转换数据框或数据框的子集为矩阵。

3.6 列表

R 中列表(list)类型来保存不同类型的数据,列表可以为 R 分析结果输出包装:例如输出一个变量,这个变量包括回归系数、预测值、残差、检验结果等等一系列不能放到规则形状数据结构中的内容,这里链表类似于 c++ 中的类。实际上,数据框也是列表的一种,但是数据框要求各列等长,而列表不要求。列表中 can 包含向量、矩阵、数据框,甚至其他列表也可以包含在其中。我们可以使用函数 `list()` 创建列表:

```
mylist <- list(object1, object2, ...)
```

其中的对象可以是本章中出现的任何结构。

我们还可以使用如下结构为列表中的对象命名:

```
mylist <- list(name1=object1, name2=object2, ...)
```

我们可以看下面的例子:

```
a <- "My first list"
b <- c(25, 26, 18, 39)
c <- matrix(1:10, nrow=5)
d <- c("one", "two", "three")
mylist <- list(title=a, ages=b, c, d)
mylist
```

运行结果如下:

```
> a <- "My first list"
> b <- c(25, 26, 18, 39)
> c <- matrix(1:10, nrow=5)
> d <- c("one", "two", "three")
> mylist <- list(title=a, ages=b, c, d)
> mylist
$title
[1] "My first list"

$ages
[1] 25 26 18 39

[[3]]
  [,1] [,2]
[1,]  1   6
[2,]  2   7
[3,]  3   8
[4,]  4   9
[5,]  5  10

[[4]]
[1] "one"  "two"  "three"
```

3.7 因子

R 中用因子代表数据中分类变量，如性别、省份、职业。有序因子代表有序量度，如打分结果，疾病严重程度等。用 `factor()` 函数把字符型向量转换成因子，例如：

```
x <- c("男", "女", "男", "男", "女")
sex <- factor(x)
sex
```

输出结果为：

```
> x <- c("男", "女", "男", "男", "女")
> sex <- factor(x)
> sex
[1] 男 女 男 男 女
Levels: 男 女
```

因子有 `class` 属性，取值为 `"factor"`，还有一个 `levels`(水平值)属性，此属性可以用 `levels()` 函数访问，如：

```
levels(sex)
```

输出结果为：

```
> levels(sex)
[1] "男" "女"
```

可以看到这个因子有两种取值：男或女，这就被称为是水平值。

用 `table()` 函数可以统计因子各水平的频数。也可以对一般的向量统计每个不同元素的出现次数，如：

```
table(sex)
```

输出结果为：

```
> table(sex)
sex
男 女
3  2
```

可以看到这个因子有两种取值的出现频数分别为：男（3 次），女（2 次）。

我们可以通过 `tapply()` 函数来按照因子分组然后每组计算另一变量的概括统计。例如下面的例子分别统计了所给数据中男女的平均身高：

```
h <- c(165, 170, 168, 172, 159)
tapply(h, sex, mean)
```

输出结果为：

```
> tapply(h, sex, mean)
      男      女
168.3333 164.5000
```

3.8 本章总结

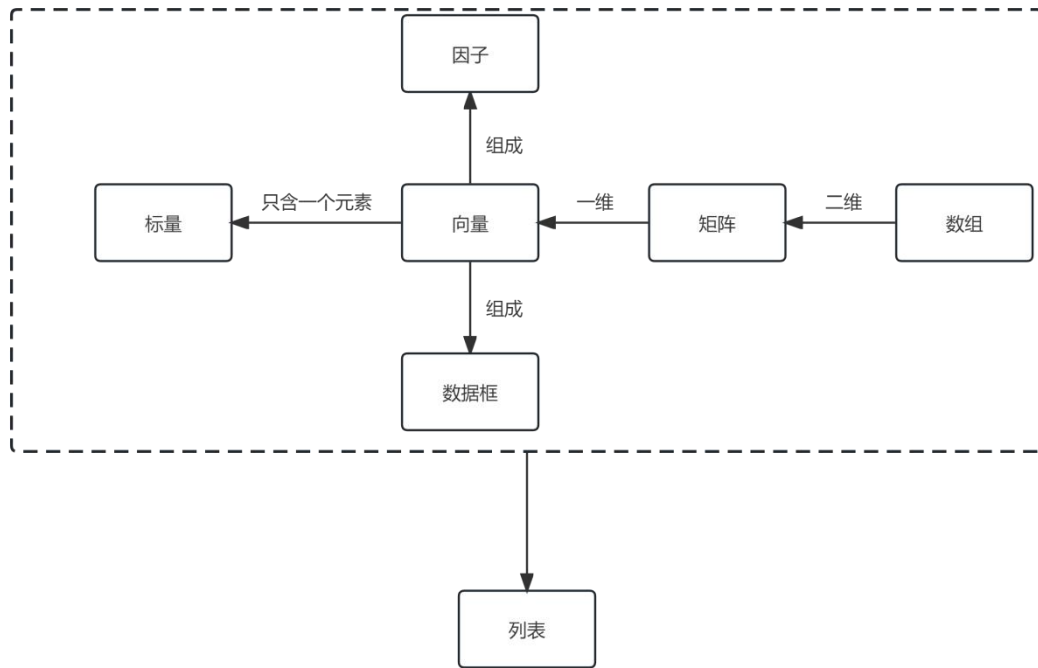


图 12 R 语言中各数据结构的关系

4 利用 R 进行基础回归分析

4.1 最小二乘回归法(OLS)

最小二乘回归法(OLS)，是通过自变量的加权和来预测连续型因变量，其中权重是通过数据估计而得到的参数。下面为 OLS 拟合模型的方程：

$$\hat{Y}_i = \hat{\beta}_0 + \hat{\beta}_1 X_{1i} + \dots + \hat{\beta}_j X_{ji} + \dots + \hat{\beta}_k X_{ki}, i = 1 \dots n$$

其中， n 为观测值的数目， k 为自变量的数目， $\hat{Y}_i$ 是第 i 次观测值对应的因变量的预测值， X_{ji} 为第 i 次观测值对应的第 j 个预测变量值， $\hat{\beta}_0$ 是截距项， $\hat{\beta}_j$ 是预测变量 j 的回归系数。在 OLS 模型中，我们需要通过减少因变量的真实值与预测值的差值来获得函数模型，即让残差平方和最小：

$$\sum_{i=1}^n (Y_i - \hat{Y}_i)^2 = \sum_{i=1}^n (\hat{Y}_i - \hat{\beta}_0 - \hat{\beta}_1 X_{1i} - \dots - \hat{\beta}_j X_{ji} - \dots - \hat{\beta}_k X_{ki})^2 = \sum_{i=1}^n \varepsilon_i^2$$

为了能够解释 OLS 拟合方程的系数，我们输入的数据要满足下列假设：

- ✧ **正态性** 对于固定的自变量值，自变量值需要成正态分布。
- ✧ **独立性** Y_i 的值之间是互相独立的。
- ✧ **线性** 因变量与自变量之间成线性相关。
- ✧ **不变方差性** 因变量的方差不随自变量的取值而改变。

在 R 中，我们使用函数 `lm()` 来拟合一个线性模型，其格式如下：

```
myfit <- lm(formula, data)
```

其中，`formula` 是要拟合的模型公式，`data` 是一个数据框，包含了用于拟合的模型信息，`myfit` 是结果对象，存储在一个列表中，包含了所拟合模型的大量信息。拟合公式 `formula` 的形式如下： $Y \sim X1 + X2 + \dots + Xk$ ，其表达式可以用下表的符号替换从而达到不同的拟合效果。

符号	用途
~	分隔符号，左边为因变量，右边为自变量
+	分隔自变量
:	表示自变量的交互项
*	表示所有可能的交互项的简洁方式，如 $x*y*z$ 可展开为 $x+y+z+x:y+x:z+y:z+x:y:z$
^	表示交互项达到某个次数，如 $(x+y+z)^2$ 可展开为 $x+y+z+x:y+x:z+y:z$
•	表示数据框内除了因变量外的所有变量的占位符，例如一个数据框中包含变量 x 、 y 、 z 和 w ，代码 $y \sim \cdot$ 表示 $y \sim x+z+w$
-	表示从方程中移除某个变量
-1	删除截距项，如 $y \sim x-1$ 拟合 y 在 x 上的回归，并强制直线通过原点
function	

表 4 R 语言表达式中常用符号

此外，我们使用下表中的函数可以获取关于拟合模型的其他信息：

函数	用途
<code>summary()</code>	展示拟合模型的详细信息
<code>coefficients()</code>	列出拟合模型的模型参数
<code>confint()</code>	提供模型参数的置信区间(默认 95%)
<code>fitted()</code>	列出拟合模型的预测值
<code>residuals()</code>	列出拟合模型的残差值
<code>anova()</code>	生成一个拟合模型的方差分析表, 或者比较两个或以上的拟合模型的方差分析表
<code>acov()</code>	列出参数模型的协方差矩阵
<code>plot()</code>	生成评估拟合模型的诊断图
<code>predict()</code>	用拟合模型对新的数据集预测因变量值

表 5 R 语言中线性回归中常用的其他函数

4.2 利用 OLS 进行简单线性回归

R 语言的基础包中的数据集中 `women` 提供了 15 位年龄在 30-39 岁之间的女性的身高体重的数据, 下面我们来通过最小二乘回归的方式来通过身高预测体重:

```
fit <- lm(weight ~ height, data=women)
summary(fit)
```

我们获得的数据总结如下:

```
> fit <- lm(weight ~ height, data=women)
> summary(fit)
```

Call:

```
lm(formula = weight ~ height, data = women)
```

Residuals:

```
      Min       1Q   Median       3Q      Max
-1.7333 -1.1333 -0.3833  0.7417  3.1167
```

Coefficients:

```
              Estimate Std. Error t value Pr(>|t|)
(Intercept) -87.51667     5.93694  -14.74 1.71e-09 ***
height       3.45000     0.09114   37.85 1.09e-14 ***
---

```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Residual standard error: 1.525 on 13 degrees of freedom

Multiple R-squared: 0.991, Adjusted R-squared: 0.9903

F-statistic: 1433 on 1 and 13 DF, p-value: 1.091e-14

从中我们可以看到所得的回归方程斜率为 3.45000, 截距为 -87.51667, 故我们利用 OLS 所得到的线性回归方程为:

$$\widehat{Weight} = 3.45000 \times Height - 87.51667$$

我们可以通过以下函数来通过画图直观地感受拟合的效果:

#通过原数据绘制散点图

```
plot(women$height, women$weight,
      xlab="Height(in inches)", ylab="Weight(in lbs)")
```

#绘制拟合的函数图像

```
lines(women$height, fitted(fit))
```

我们获得的图像如下图所示：

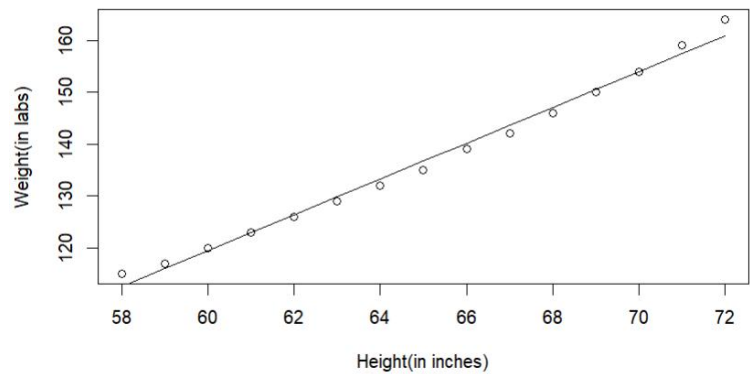


图 13 拟合函数的图像

从中，我们可以看出所得的回归方程基本上符合了这些点的走势，但从中我们也可以看到两端的点距离这条直线的偏离较大，下面我们通过 `plot()` 函数生成评估拟合模型的诊断图来进一步评估拟合的效果：

`plot(fit)`

我们获得的图像如下图所示：

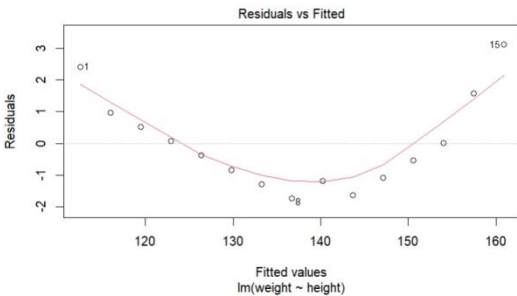


图 14 残差-拟合图

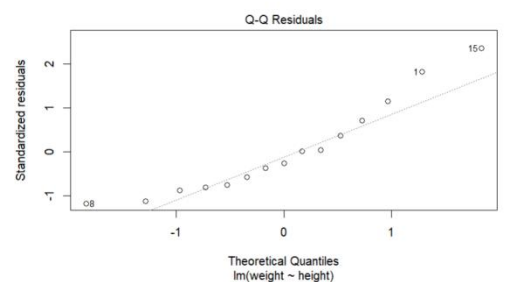


图 15 正态 Q-Q 图

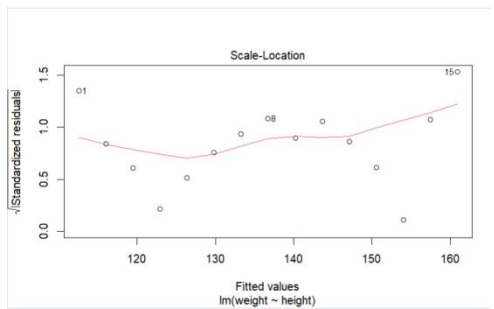


图 16 位置-尺度图

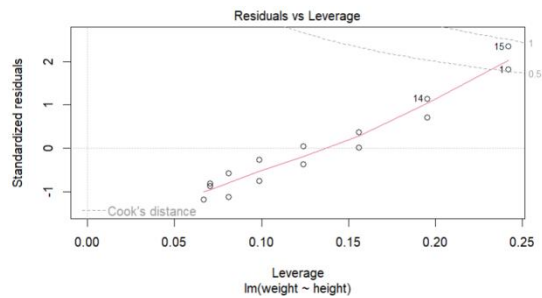


图 17 残差-杠杆图

图 14 是残差-拟合图，从中我们可以看到每个数据相较于拟合曲线的方差，若因变量与自变量线性相关，则残差值与拟合值不能有系统关联，但从图 14 中，我们可以看到这是一条曲线，说明我们应该加上一个二次项来更好地拟合；图 15 是正态 Q-Q 图，若满足正态性，则图上的点应落在 45° 的直线上，显然图 15 符合；图 16 是位置-尺度图，若满足同方差性，则水平线周围的点应该随机分布，该图是符合的；图 17 是残差-杠杆图，从中可以识别出离群点、高杠杆点和强影响点。

4.3 利用 OLS 进行多项式回归

在 4.2 中对拟合模型的分析中，我们可以看到，我们可以添加一个二次项来提高回归的精度，利用下面的代码我们可以拟合一个含有二次项的方程：

```
fit <- lm(weight ~ height + I(height^2), data=women)
summary(fit)
```

我们获得的数据总结如下：

```
> fit <- lm(weight ~ height + I(height^2), data=women)
> summary(fit)
```

Call:

```
lm(formula = weight ~ height + I(height^2), data = women)
```

Residuals:

Min	1Q	Median	3Q	Max
-0.50941	-0.29611	-0.00941	0.28615	0.59706

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	261.87818	25.19677	10.393	2.36e-07	***
height	-7.34832	0.77769	-9.449	6.58e-07	***
I(height^2)	0.08306	0.00598	13.891	9.32e-09	***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.3841 on 12 degrees of freedom

Multiple R-squared: 0.9995, Adjusted R-squared: 0.9994

F-statistic: 1.139e+04 on 2 and 12 DF, p-value: < 2.2e-16

从中我们可以看到所得的回归方程的二次项系数为 0.08306，一次项系数为 -7.34832，截距为 261.87818，故我们利用 OLS 所得到的线性回归方程为：

$$\widehat{Weight} = 0.083 \times Height^2 - 7.34832 \times Height + 261.87818$$

我们可以通过以下函数来通过画图直观地感受拟合的效果：

#通过原数据绘制散点图

```
plot(women$height, women$weight,
      xlab="Height(in inches)", ylab="Weight(in lbs)")
```

#绘制拟合的函数图像

```
lines(women$height, fitted(fit))
```

我们获得的图像如下图所示：

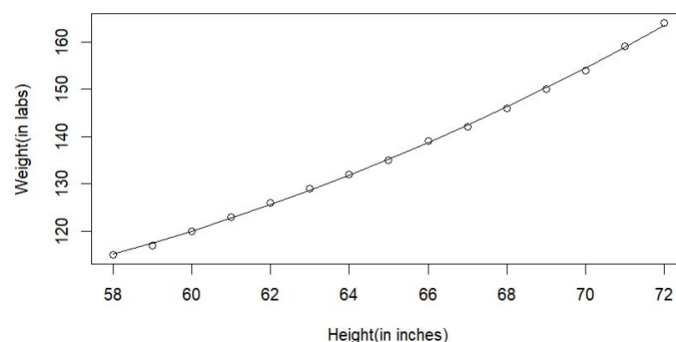


图 18 添加二次项后拟合函数的图像

通过图像我们可以看出，此时线性回归模型的拟合程度相较于 4.2 中提升了很多，并且我们可以从获得的数据中看出，此时模型的方差解释率已经到了 99.9%，我们下面再通过残差-拟合图来观察一下拟合的效果：

`plot(fit)`

我们获得的图像如下图所示：

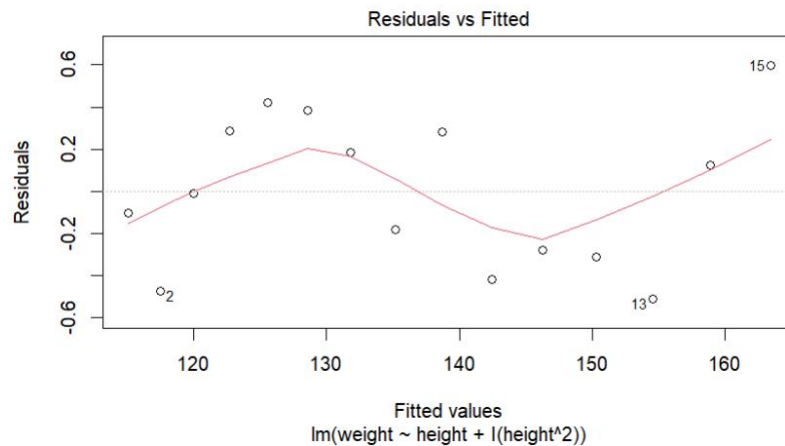


图 19 残差-拟合图

此时，残差的分布更加均匀，表明拟合的效果相较于之前更好。

4.4 利用 OLS 进行带交互项的多元线性回归

下面我们通过 R 语言基础包中的 `mtcars` 数据框来利用 OLS 进行含有交互项的多元线性回归：

```
fit <- lm(mpg ~ hp + wt + hp:wt, data=mtcars)
summary(fit)
```

我们获得的数据总结如下：

```
> fit <- lm(mpg ~ hp + wt + hp:wt, data=mtcars)
> summary(fit)
```

Call:
lm(formula = mpg ~ hp + wt + hp:wt, data = mtcars)

Residuals:

	Min	1Q	Median	3Q	Max
	-3.0632	-1.6491	-0.7362	1.4211	4.5513

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	49.80842	3.60516	13.816	5.01e-14	***
hp	-0.12010	0.02470	-4.863	4.04e-05	***
wt	-8.21662	1.26971	-6.471	5.20e-07	***
hp:wt	0.02785	0.00742	3.753	0.000811	***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.153 on 28 degrees of freedom
Multiple R-squared: 0.8848, Adjusted R-squared: 0.8724
F-statistic: 71.66 on 3 and 28 DF, p-value: 2.981e-13

从 `Pr` 一栏中我们可以看到，`hp` 与 `wt` 的交互项对因变量的影响是显著的，这说

明因变量与其中一个自变量的关系依赖于另一个自变量，我们可以通过 `effect` 包中的 `effect()` 函数来用图形展示交互项的结果，其用法为：

```
plot(effect(term, mod,, xlevels), multiline=TRUE)
```

其中，`term` 为模型要绘制的项，`mod` 为通过 `lm()` 拟合的模型，`xlevels` 是一个列表，指定变量要设置的常量值，`multiline=TRUE` 表示添加相应的直线，`lines` 选项指定每条线的线条类型（1 为实线，2 为虚线，3 为点线），例如：

```
library(effects)
plot(effect("hp:wt", fit,, list(wt=c(2.2, 3.2, 4.2))),
      lines=c(1, 2, 3), multiline=TRUE)
```

我们获得的图像如下图所示：

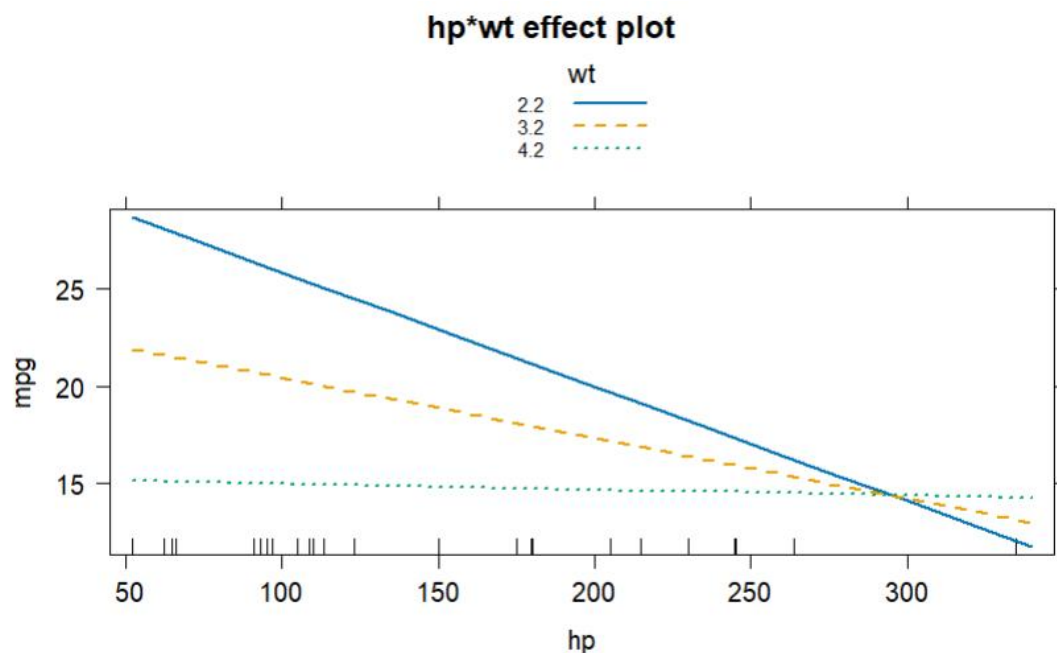


图 20 hp 与 wt 的交互项图线

从中，我们可以看到随 `wt` 的取值不同，其对 `hp` 与 `mpg` 的关系影响也不同，当 `wt=4.2` 时，`hp` 对 `mpg` 没有影响。

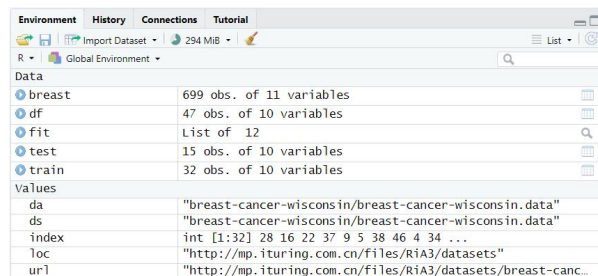
5 在 R 中通过机器学习技术生成预测模型

5.1 数据准备

在本章的生成预测模型中所用的数据集为威斯康星州的乳腺癌数据，其中包含 699 个观测值，458 个为良性观测值，241 个为恶性观测值，数据集中包含的变量有：ID、肿块厚度、细胞大小的均匀性、细胞形状的均匀性等，下面的代码进行了数据的准备：

```
loc <- "http://mp.ituring.com.cn/files/RiA3/datasets"
ds <- "breast-cancer-wisconsin/breast-cancer-wisconsin.data"
url <- paste(loc, ds, sep="/")
breast <- read.table(url, sep=",", header=FALSE, na.strings="?")
names(breast)<-c("ID", "clumpThickness", "sizeUniformity",
                "shapeUniformity", "singleEpithelialCellSize",
                "bareNuclei", "blandChromatin",
                "normalNucleoli", "mitosis", "class")
df <- breast[-1]
df$class <- factor(df$class, levels=c(2,4),
                  labels=c("benign", "malignant"))
df <- na.omit(df)
set.seed(1234)
index <- sample(nrow(df), 0.7*nrow(df))
train <- df[index,]
test <- df[-index,]
table(train$class)
table(test$class)
```

上面的代码首先定义了数据集的存储位置（loc）和数据集的文件名（da）；然后，使用 paste 函数将这两个部分拼接成完整的 URL；使用 read.table 函数从 URL 下载数据，设置分隔符为逗号，不包含列名，将"?"视为缺失值并为数据框 breast 设置列名。删除第一行（因为第一行是列名，不是实际的数据），将类别变量（class）转换为因子类型，并指定类别级别为 2 和 4，标签分别为"benign"和"malignant"。使用 na.omit 函数删除包含缺失值的行，设置随机数种子为 1234，以确保结果的可重复性。使用 sample 函数随机抽取 70% 的数据作为训练集，剩余的 30% 作为测试集；最后，打印出训练集和测试集的分类分布。我们可以在环境中看到我们已经导入的数据，其中训练集有 32 个对象，测试集有 15 个对象：



Environment	History	Connections	Tutorial
R - Global Environment			
Data			
breast		699 obs. of 11 variables	
df		47 obs. of 10 variables	
fit		List of 12	
test		15 obs. of 10 variables	
train		32 obs. of 10 variables	
Values			
da		"breast-cancer-wisconsin/breast-cancer-wisconsin.data"	
ds		"breast-cancer-wisconsin/breast-cancer-wisconsin.data"	
index		int [1:32] 28 16 22 37 9 5 38 46 4 34 ...	
loc		"http://mp.ituring.com.cn/files/RiA3/datasets"	
url		"http://mp.ituring.com.cn/files/RiA3/datasets/breast-canc..."	

图 21 环境中已经导入的数据

5.2 Logistic 回归

Logistic 回归是广义线性模型中的一种，可以根据一组数值变量预测二元输出，R 中的基本函数 `glm()` 可以拟合 Logistic 模型，下面的代码展示了利用 5.1 中的数据进行 Logistic 回归的过程：

```
#拟合 Logistic 回归
fit <- glm(class~., data=train, family=binomial())
#检查模型
summary(fit)
#对训练集外观测值进行分类
prob <- predict(fit, test, type="response")
logit.pred <- factor(prob>.5, levels=c(FALSE,TRUE),labels=c("benign",
"malignant"))
#评估预测的准确度
logit.perf <- table(test$class, logit.pred,
dnn=c("Actual", "Predicted"))
logit.perf
```

下面的数据展示了训练的模型对测试集的准确度：

	Predicted	
Actual	benign	malignant
benign	10	3
malignant	1	1

可以看到 15 个测试集的数据中正确分类了 11 个数据，准确度达到了 73.3%，高于不加训练的最坏的情况，训练是成功的。

5.3 决策树

决策树是数据挖掘领域的常用模型，它的基本思想是对自变量进行二元分离，从而构造出一棵可以用于预测新观测值所属类别的树。经典决策树以一个二元输出变量(对应威斯康星州乳腺癌数据集中的良性/恶性)和一组自变量(对应 9 个细胞特征)为基础。具体算法如下。

(1)为达到最大化两类纯度，选取一个最佳自变量将所有观测值划分为两类。若自变量为连续型，则确定一个分割点进行分类；若自变量为分类型，则对各类别进行合并，以形成具有最大纯度的两个类别。

(2)在将数据划分为两类后，针对每个子类别重复步骤(1)。

(3)持续执行步骤(1)和步骤(2)，直至子类别中的观测值数量少于最小观测值标准，或无法通过分类方法将不纯度降低至给定值以下。最终得到的子类别即为终端节点。根据终端节点中观测值类别数的众数，判断该终端节点的所属类别。

(4)在进行观测值分类时，将观测值的属性从根节点输入，然后传递至终端节点。最后，将终端节点的因变量值赋予观测值，即可确定观测值所属的类别。

但是，上述算法会得到一棵过大的数剪枝是决策树学习算法对付"过拟合"的主要手段。在决策树学习中，为了尽可能正确分类训练样本，结点划分过程将不断重复，有时会造成决策树分支过多，这时就可能因训练样本学得"太好"了，以致于把训练集自身的一些特点当作所有数据都具有的一般性质而导致过拟合。因此，可通过主动去掉一些分支来降低过拟合的风险。

我们可以使用 `rpart` 包中的函数 `rpart()` 来创建决策树，函数 `prune()` 可以对决策树进行剪枝。下面的代码演示了如何对 5.1 中的数据进行生成决策树的操作：

```
#生成决策树
library(rpart)
dtree <- rpart(class~., data=train, method="class",
               parms=list(split="information"))
dtree$cptable
plotcp(dtree)
dtree.prund <- prune(dtree, cp=1.5)
#剪枝
dtree.pred <- predict(dtree.prund, test, type="class")
dtree.perf <- table(test$class, dtree.pred,
                   dnn=c("Actual","Predicted"))
dtree.perf
```

下面的数据展示了训练的模型对测试集的准确度：

	Predicted	
Actual	benign	malignant
benign	13	0
malignant	2	0

可以看到 15 个测试集的数据中正确分类了 13 个数据，准确度达到了 86.7%，高于不加训练的最坏的情况，训练是成功的，但是我们需要注意的是关于 `malignant` 的分类全部错误，我们还需要进一步参数调优并且扩大数据集进行进一步训练。

5.4 支持向量机

支持向量机(support vector machines,SVM)是一类可以用于分类和回归的监督机器学习模型，可以输出较准确的预测结果。

SVM 可以通过 R 中的 `kernlab` 包中的 `ksvm()` 函数和 `e1071` 包中的 `svm()` 函数实现，下面的代码就实现了基于 SVM 建立模型：

```
library(e1071)
set.seed(1234)
fit.svm <- svm(class~., data=train)
fit.svm
svm.pred <- predict(fit.svm, test)
svm.perf <- table(test$class, svm.pred, dnn=c("Actual","Predicted"))
svm.perf
```

下面的数据展示了训练的模型对测试集的准确度：

	Predicted	
Actual	benign	malignant
benign	13	0
malignant	1	1

可以看到 15 个测试集的数据中正确分类了 14 个数据，准确率比上面两种方法更高了一些，准确度达到了 93.3%，高于不加训练的最坏的情况，训练是成功的。

6 写在最后

本次的新生研讨课的课程报告我选择了自学一门技术,选择了 R 语言这一技术,并且初步接触了回归分析、机器学习等的这些内容,但是在大一上这短短的一段时间对于此的自学,对于一些内容或许还仅仅是知道一些皮毛,对于其中蕴含的数学原理以及计算机原理了解还不算很多,这些都需要在后续的学习中进一步掌握。但总的来说,通过这一段时间的学习,还是挺大的锻炼了自己的能力,受益匪浅。

参考资料

[01] Robert I.Kabacoff. R 语言实战[M]. 第 3 版.

[02] 周志华. 机器学习[M].

[03] 李东风. R 语言教程.

https://www.math.pku.edu.cn/teachers/lidf/docs/Rbook/html/_Rbook/index.html